

Case Técnico — CRI Soluções Imobiliárias

SISTEMA DE GESTÃO E QUALIFICAÇÃO DE LEADS

Dashboard + Automação + Inteligência Artificial

DESENVOLVIDO POR:

Thaissa Kailaine

Tecnologias: React · TypeScript · Tailwind CSS ·
Supabase · n8n · Gemini

Interface: Vercel

Automação: n8n hospedado em VPS

BANCO DE DADOS E MODELAGEM DOS LEADS

O primeiro desafio foi entender que o problema não era simplesmente cadastrar um lead. Pensei no banco de dados como o coração do sistema, então além dos campos obrigatórios do case, como nome, telefone, origem, status e texto de interesse, estruturei **também região de interesse, tipo de imóvel, características desejadas, faixa de valor, prazo de compra, intenção de compra e dados ainda faltantes para qualificação, tirando dúvida com o Caio, gestor de tecnologia da CRI, para entender o que caracteriza um lead qualificado e um lead perdido na prática para ficar mais próximo da realidade da CRI.** Resumo comercial, próxima ação e uma mensagem personalizada. A ideia era evitar que o sistema simplesmente guardasse o texto original do cliente, e sim transformasse isso em **informação realmente útil** para a operação comercial.

Escolhi Supabase por já ter familiaridade com a tecnologia de um projeto anterior, e porque ele oferece Postgres hospedado na nuvem, interface administrativa intuitiva e baixo custo inicial, o que me deixou livre para focar na lógica de negócio em vez de montar uma infraestrutura de banco do zero.

A dificuldade mais interessante aqui foi perceber que a IA podia interpretar corretamente o significado de uma informação, mas ainda assim gerar representações diferentes para o mesmo dado, principalmente em regiões e faixas de valor. Não era a IA errando, era falta de uma estrutura controlada entre a interpretação e o armazenamento. Resolvi separando claramente três etapas, interpretação da IA, normalização e persistência, o que deixou os dados muito mais consistentes e os filtros mais confiáveis.

Com mais tempo, eu automatizaria também a transição de status para perdido direto na camada de dados, hoje essa regra existe só na aplicação.

AUTOMAÇÃO E AGENTE DE IA

O webhook funciona como ponto de entrada do sistema, recebendo o lead e disparando o processamento. A partir daí, o agente de IA, usando Gemini, interpreta o texto livre enviado pelo cliente e **extrai região, tipo de imóvel, faixa de valor, prazo de compra e características, além de classificar a intenção de compra em alta, média, pesquisando ou não identificada**. Depois disso os dados normalizados são salvos no Supabase, e o agente também gera um **resumo comercial, sugere a próxima ação e monta uma primeira mensagem personalizada**, que muda de tom dependendo da **origem do lead**, mais profissional para quem veio do site, mais natural para WhatsApp, mais acolhedora para indicação.

Usei n8n para orquestrar tudo porque o fluxo precisava **ser orientado por eventos, e essa estrutura também deixa o sistema preparado para receber leads de outros canais no futuro**, como Instagram ou formulários do site, mantendo a lógica de processamento centralizada. Optei pelo Gemini para interpretar a linguagem natural dos leads, mas sem tratar a IA como uma caixa preta que decide tudo, a aplicação continua controlando como essas informações são validadas e usadas.

Aqui apareceram algumas das dificuldades mais técnicas do projeto. A primeira foi **idempotência**, já que em integrações baseadas em webhook a mesma requisição pode chegar **duplicada**, o que geraria dois registros para o mesmo lead. Criei uma **chave de idempotência** para isso, e testei enviando a mesma chave duas vezes propositalmente, o primeiro lead foi salvo normalmente e o segundo foi **bloqueado**, confirmando que o comportamento estava correto.

A segunda foi disponibilidade, baseada numa experiência anterior em que um **agente de IA ficou temporariamente indisponível por um pico de requisições**, então configurei **retry na automação**, com três tentativas e intervalo de dois segundos, para que uma falha temporária de comunicação não derrubasse todo o processamento.

AUTOMAÇÃO E AGENTE DE IA

Também precisei ajustar validações depois de identificar, em testes, que o **campo de texto de interesse podia chegar vazio**, então passei a impedir o envio sem as informações mínimas de **nome, telefone e texto de interesse**.

Com mais tempo, eu **implementaria uma fila de processamento para lidar melhor com volume alto de leads, e conectaria o fluxo direto ao WhatsApp**, já que tenho facilidade com Evolution API, automatizando o recebimento dos contatos até o momento em que a intervenção humana se torna necessária.

INTERFACE E DASHBOARD

O dashboard foi pensado para dar uma visão rápida da carteira, com indicadores de total de leads, qualificados, em contato e perdidos, além de gráficos de distribuição e qualificação por origem, listagem com filtros, modal de detalhes por lead e simulação de novos leads.

Uma **regra que criei durante o desenvolvimento foi de acompanhamento**, leads entre **sete e nove dias sem contato entram em atenção**, e com **dez dias ou mais são considerados perdidos**, o que transforma um dado simples, a data do último contato, em algo que **o consultor pode agir sobre antes de perder a oportunidade**.

Escolhi React com TypeScript e Tailwind em vez de uma ferramenta de geração automática de interface **porque queria mais controle sobre a estrutura dos componentes, a integração com APIs e a evolução futura do sistema, pensando num cenário de milhares de leads e recursos como paginação e carregamento sob demanda**. TypeScript também ajudou a manter contratos mais claros entre os dados vindos do banco e os componentes da tela.

A dificuldade aqui foi garantir que os filtros trabalhassem de fato com os dados armazenados, precisei revisar a diferença entre valor armazenado, valor apresentado e valor usado no filtro, o que reforçou uma decisão importante do projeto, a camada de apresentação não

INTERFACE E DASHBOARD

deve corrigir dados inconsistentes, isso é papel do banco e do processamento. Também precisei ajustar o fluxo de atualização para que, depois de um novo lead processado pela automação, o dashboard refletisse isso imediatamente.

Com mais tempo, criaria uma estrutura histórica para alimentar um modelo de IA com os dados de interesse dos leads ao longo do tempo, **identificando tendências de região, tipo de imóvel e faixa de valor.**

CONSULTA ANALÍTICA E RECOMENDAÇÕES COMERCIAIS

Para responder às perguntas da etapa de interpretação de dados, usei **duas views SQL direto no Supabase**, sobre os leads que eu mesma cadastrei durante os testes.

Qual origem gerou mais leads, o WhatsApp, com folga, concentrando a maior parte da carteira. A consulta usada foi:

```
CREATE VIEW public.leads_por_origem AS SELECT origem, COUNT(*) AS total_leads
FROM public.leads
GROUP BY origem
ORDER BY total_leads DESC;
```

Qual o percentual de qualificados por origem, Indicação teve a maior taxa, 40%, com 2 de 5 leads qualificados, seguida por WhatsApp com 36,8%, 7 de 19, e Site com 0%, nenhum dos 5 leads qualificado ainda. A consulta usada foi:

```
CREATE VIEW public.percentual_qualificados_por_origem AS SELECT
origem, COUNT(*) AS total_leads,
COUNT(*) FILTER (WHERE status = 'qualificado') AS total_qualificados,
ROUND(COUNT(*) FILTER (WHERE status = 'qualificado') * 100.0 /
COUNT(*), 2) AS percentual_qualificados
FROM public.leads
GROUP BY origem
ORDER BY percentual_qualificados DESC;
```

Um padrão relevante que observei foi que o **WhatsApp tem simultaneamente o maior volume e uma das maiores taxas de qualificação, o que sugere que esse canal traz leads com perfil mais aderente ao negócio, e não só mais quantidade**. Também identifiquei concentração de demanda por região e tipo de imóvel, como Balneário Camboriú combinado com apartamento, que aparece com frequência bem maior que as outras combinações.

CONSULTA ANALÍTICA E RECOMENDAÇÕES COMERCIAIS

Os indicadores do dashboard **não são valores fixos**, eles são calculados a partir dos **dados reais do Supabase**, usando essas mesmas views. A partir desses cálculos, **cruzei informações na aplicação**, como região combinada com tipo de imóvel, e origem combinada com taxa de qualificação, **para gerar recomendações que ajudam o consultor a priorizar onde existe maior concentração de demanda qualificada**, sem que o sistema decida automaticamente onde investir, ele só oferece o sinal, a decisão final continua sendo da equipe.

Com mais tempo, integraria a solução a um **catálogo real de imóveis**, **cruzando perfil do lead**, orçamento e região com o inventário disponível, para a IA sugerir imóveis compatíveis sem nunca inventar disponibilidade.

MEU PAPEL NO DESENVOLVIMENTO E USO DE IA

Usei IA com agente Gemini apoiado por um arquivo de instruções CLAUDE.md, **principalmente para acelerar implementação de componentes, ajustes de interface e refatorações**. A parte visual também teve apoio de IA, **usei o Lovable** como referência inicial e depois criei um **documento próprio de identidade visual baseado na presença digital da CRI**, para orientar o desenvolvimento. Mas as **decisões de arquitetura, regras de negócio, estrutura dos dados, validações e critério de qualificação foram sempre minhas**, e quando a IA gerava algo tecnicamente funcional mas fora da regra de negócio, eu revisava o comportamento e ajustava a instrução, não simplesmente aceitava o resultado.

TESTES

Testei o fluxo completo com cerca de quarenta leads, cobrindo criação, campos obrigatórios, idempotência, extração e classificação de dados, atualização do dashboard, filtros e alertas de acompanhamento, além de testes destrutivos, apagando registros e reprocessando para garantir que o sistema continuava funcionando depois.

ACESSO À SOLUÇÃO

Aplicação:

<https://cri-leads-eight.vercel.app/>

Repositório:

<https://github.com/Tkailaine/crileads>

Documentação mais detalhada sobre o desenvolvimento e com imagens:

<https://github.com/Tkailaine/crileads/blob/main/docs/material-complementar.pdf>